4.1.2018

EE 241 DIGITAL CIRCUITS
FINAL EXAM SOLUTION MANUAL

Name:

Q1) (15 points) Design and implement a combinational circuit (in dataflow modeling) having the input output relation (in terms of decimal numbers) as follows.

input	output
0	0
1	0
2	1
3	1
4	2
5	2
6	3
7	3

Your input will be a three-bit array i[2:0]. Your output will be a two-bit array o[1:0].

Note: No partial points will be given in this question.

SOLUTION:

Input i[2] i[1] i[0]	Output o[1] o[0]
0 0 0	0 0
0 0 1	0 0
0 1 0	0 1
0 1 1	0 1
1 0 0	1 0
1 0 1	1 0
1 1 0	1 1
1 1 1	1 1

```
module first_question(o,i);

input [2:0] i;
output [1:0] o;

assign o[1]= i[2];
assign o[0]=i[1];
endmodule
```

```
`timescale 1ns / 1ps

module first_question_tb;

// Inputs
reg [2:0] it;

// Outputs
wire [1:0] ot;

// Instantiate the Unit Under Test (UUT)
first_question UUT (.o(ot),.i(it));

//Providing input to the UUT
initial begin
// Initialize Inputs
it = 3'b000;

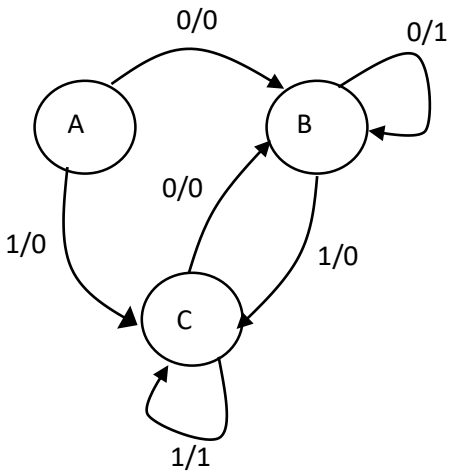
// Wait 100 ns for global reset to finish
#100;

// Add stimulus here
repeat (7)
#100 it = it + 1'b1;
end

//Display the result on the Tcl console (Optional)
initial  begin
$display("   input    output");
$monitor("\t%b \t%b \t%b \t%b \t%b",it[2], it[1],it[0],ot[1],ot[0]);
end

endmodule
```

Q2) (15 points) Represent the below state machine as a module in Verilog. Input signals of the module are “clk” (clock), “clr” (clear), and “i” (one bit value). Output of the module is a one bit signal “o”. Initial state of the machine is A which is set every time clr signal has logic level 1.



Note: No partial points will be given in this question.

SOLUTION:

```
module second_question(y,x,clk,clr);

input x,clk,clr;
output reg y;

reg [1:0] state=2'b00;
parameter A=2'b00, B=2'b01, C=2'b10;

always @ (posedge clk)
begin
if (clr == 1) state <= A;
else
begin
if (x == 0)
  case (state)
    A : state <= B;
    B : state <= B;
    C : state <= B;
   endcase
else
  case (state)
    A : state <= C;
    B : state <= C;
    C : state <= C;
```

```
  endcase
if ((x == 0) && (state == B)||(x == 1) && (state == C)) y <= 1;
else y <= 0;
end
end
endmodule


`timescale 1ns / 1ps

module second_question_tb;

reg x,clk,clr;
wire y;

second_question UUT(y,x,clk,clr);

initial begin
#2 clk=0;
forever
#2 clk=~clk;
end

initial
begin
x = 1'b0;
clr = 1'b1;
#1 clr = 1'b0;

#5 x = 1'b1;
#5 x = 1'b0;
#5 x = 1'b1;
#5 x = 1'b1;
#5 x = 1'b0;
#5 x = 1'b1;
#5 x = 1'b0;

#5 clr = 1'b1;
#5 clr = 1'b0;
end
endmodule
```

Q3) (30 points) Design a simple display unit in Verilog on the Basys3 board with the following specifications.

i. **(10 points)** Every time the central button (btnC) on the Basys3 board is pressed, LEDs on the board turn on one by one. As an example, when the btnC is pressed first time, the LED[0] will turn on; when the button is pressed again, both LED[0] and LED[1] will turn on. This successive turning on will continue by the following btnC presses.

ii. **(10 points)** When all the LEDs (LED[0] to LED[15]) are turned on and the btnC is pressed again, all the LEDs except LED[0] will turn off and the rightmost seven segment display will show the turn number. Hence, when the first turn (turning on LEDs from LED[0] to LED[15]) has finished, the display will show 1, and it will increase by successive turns. The display can show numbers from 0 to 9. Initially, the display will show the number 0.

iii. **(10 points)** We will update the previous part such that, the maximum LED to be turned on will be decided by the switches on the Basys3 board. For example, when the sw[9] is "on", turning on the LEDs will continue till LED[9] (including it). Then the new turn will start. To note here, more than one switch can be turned on. Your system will pick the one with the highest location. As an example, if both switches sw[9] and sw[12] are "on", the turn will continue till LED[12] (including it.)

SOLUTION:

```
module third_question(sw,clk,btnC,led,seg,an);
input [15:0] sw;
input clk,btnC;
output reg [15:0] led=16'd0;
output reg [6:0] seg=7'b1000000;
output reg [3:0] an=4'b1110;
reg btnC_reg=1'b0;
reg [3:0] cnt=4'd0,maxc=4'd15;

always @(posedge clk)
begin
   btnC_reg=btnC;


//    if (sw[15]) maxc=4'd15;
//    else if (sw[14]) maxc=4'd14;
//    else if (sw[13]) maxc=4'd13;
//    else if (sw[12]) maxc=4'd12;
//    else if (sw[11]) maxc=4'd11;
//    else if (sw[10]) maxc=4'd10;
//    else if (sw[9])  maxc=4'd9;
//    else if (sw[8])  maxc=4'd8;
//    else if (sw[7])  maxc=4'd7;
//    else if (sw[6])  maxc=4'd6;
```

```
//    else if (sw[5])  maxc=4'd5;
//    else if (sw[4])  maxc=4'd4;
//    else if (sw[3])  maxc=4'd3;
//    else if (sw[2])  maxc=4'd2;
//    else if (sw[1])  maxc=4'd1;
//    else if (sw[0])  maxc=4'd0;


    casex(sw)
    16'b1xxxxxxxxxxxxxxx:  maxc<=4'd15;
    16'b01xxxxxxxxxxxxxx:  maxc<=4'd14;
    16'b001xxxxxxxxxxxxx:  maxc<=4'd13;
    16'b0001xxxxxxxxxxxx:  maxc<=4'd12;
    16'b00001xxxxxxxxxxx:  maxc<=4'd11;
    16'b000001xxxxxxxxxx:  maxc<=4'd10;
    16'b0000001xxxxxxxxx:  maxc<=4'd9;
    16'b00000001xxxxxxxx:  maxc<=4'd8;
    16'b000000001xxxxxxx:  maxc<=4'd7;
    16'b0000000001xxxxxx:  maxc<=4'd6;
    16'b00000000001xxxxx:  maxc<=4'd5;
    16'b000000000001xxxx:  maxc<=4'd4;
    16'b0000000000001xxx:  maxc<=4'd3;
    16'b00000000000001xx:  maxc<=4'd2;
    16'b000000000000001x:  maxc<=4'd1;
    16'b0000000000000001:  maxc<=4'd0;
    endcase


    case(cnt)
    0:  seg = 7'b1000000; // 0
    1:  seg = 7'b1111001; // 1
    2:  seg = 7'b0100100; // 2
    3:  seg = 7'b0110000; // 3
    4:  seg = 7'b0011001; // 4
    5:  seg = 7'b0010010; // 5
    6:  seg = 7'b0000010; // 6
    7:  seg = 7'b1111000; // 7
    8:  seg = 7'b0000000; // 8
    9:  seg = 7'b0010000; // 9
    endcase
end

always @(posedge btnC_reg)
begin
```

```
   if(sw==16'd0) led=led;
   else if(led[maxc])
   begin
      led=16'd1;
      if (cnt==4’d9) cnt=4’d0;
      else cnt=cnt+1’b1;
   end
   else
      led=2*led+1;
end
endmodule


`timescale 1ns / 1ps

module third_question _tb;
reg [15:0] sw_t;
reg clk_t,btnC_t;
wire [15:0] led_t;
wire[6:0] seg_t;
wire[3:0] an_t;

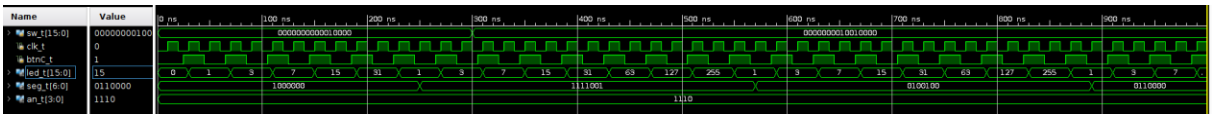
third_question u1(sw_t,clk_t,btnC_t,led_t,seg_t,an_t);

initial
begin
clk_t=1'b0;
forever
#10 clk_t=~clk_t;
end

initial
begin
btnC_t=1'b0;#5;
repeat (64)
#20 btnC_t=~btnC_t;
end

initial
begin
sw_t=16'b0000000000010000; #300;
sw_t=16'b0000000010010000;
end
```

endmodule


Name
Value
sw_t[15:0]
clk_t
btnC_t
led_t[15:0]
seg_t[6:0]
an_t[3:0]
0000000010000
0
1
15
0110000
1110
0 ns
100 ns
200 ns
300 ns
400 ns
500 ns
600 ns
700 ns
800 ns
900 ns
0000000000010000
0000000001001000
0 1 3 7 15 31 1 3 7 15 31 63 127 255 1 3 7 15 31 63 127 255 1 3 7
1000000
1111001
0100100
0110000
1110

Q4) (40 points) We will design a simple data receiver system with below specifications. To make the system more modular, represent each part below by a standalone submodule.

i. **(5 points)** The incoming bits will be read from the rx input (pin). When no data is transmitted, rx will be at logic level 1. The reception operation starts when rx goes to logic level 0. This will be indicated by the start bit (which is initially at logic level 0). The layout of this module will be as follows.

```
module start_reception(start, clk, rx);
input clk; //initialy the frequency of this clock is 100 MHz.
input rx;
output reg start;
```

ii. **(10 points)** If the start bit is set by the start_reception module, the new clock frequency will be 115200 Hz. The layout of this module will be as follows.

```
module freq_divider(start, clk, clkn);
input clk; //initialy the frequency of this clock is 100 MHz.
input start;
output reg clkn;
```

iii. **(15 points)** If the start bit is set by the start_reception module, and the new clock signal is generated, then the receive module will gather 9 bits from the rx input by each rising edge of clkn signal (the most significant bit being the first received bit). As the 9 bits are received, the done signal will turn on. Hence, no more data reception will be done. Initially, the done signal is at logic level 0. The layout of this module will be as follows.

```
module receive(start, clkn, rx, bits, done);
input clkn;
input start;
input rx;
output reg [8:0] bits;
output reg done;
```

iv. **(10 points)** The received 9 bits in the receive module consist of 8 bits representing an ASCII character (most significant bit being the first received bit), and one parity bit. If the done signal is set, transmission is enabled. The odd parity is used in transmission. If the received 9 bits are valid according to the parity check and the received character is “O”, the LED[0] on the Basys3 board will turn on. If the received 9 bits are valid according to the parity check and the received character is “C” the LED[0] on the Basys3 board will turn off. The layout of this module will be as follows.

```
module LED_turn_on_off(clk, bits, done, led);
input clk; //initialy the frequency of this clock is 100 MHz.
input [8:0] bits;
input done;
output reg [0:0] led;
```

SOLUTION:

Part 1

```
module start_reception(start, clk, rx);
input clk; //initialy the frequency of this clock is 100 MHz.
input rx;
output reg start;

initial
start=1'b0;

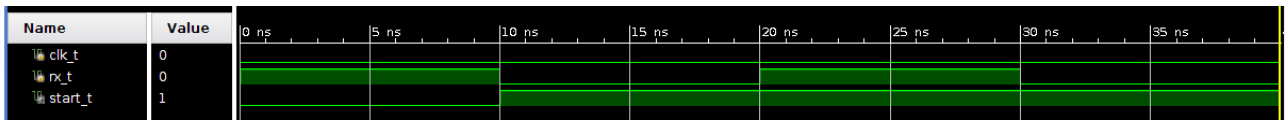
always @(rx)
   if (~rx)
      start=1'b1;
endmodule


`timescale 1ns / 1ps

module start_reception_tb;
reg clk_t;
reg rx_t;
wire start_t;

start_reception u1(start_t, clk_t, rx_t);

initial
begin
clk_t=1'b0;
rx_t=1'b1; #10;
rx_t=1'b0; #10;
rx_t=1'b1; #10;
rx_t=1'b0; #10;
$stop;
end
endmodule
```



Part 2

```
module freq_divider(start, clk, clkn);
input clk;
input start;
output reg clkn;
reg [8:0] counter;

initial
begin
clkn=1'b0;
counter<=9'd0;
end

always @(posedge clk)
begin
   if (start==1'b1)
   begin
      if(counter==9'd433)
      begin
         counter<=9'd0;
         clkn <= ~clkn;
      end
      else
         counter<=counter+1'b1;
   end
   else
   begin
      counter<=9'd0;
      clkn <= 1'b0;
   end
end
endmodule

`timescale 1ns / 1ps

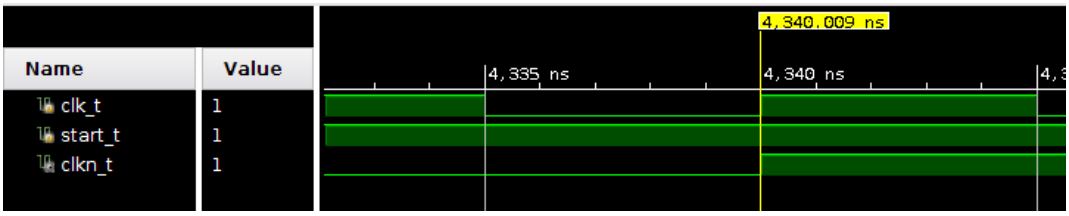
module freq_divider_tb;
reg clk_t;
reg start_t;
wire clkn_t;

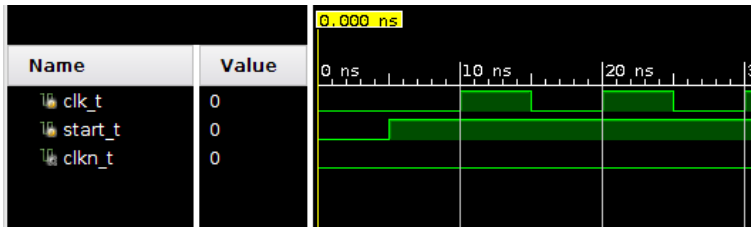
freq_divider u1(start_t, clk_t, clkn_t);

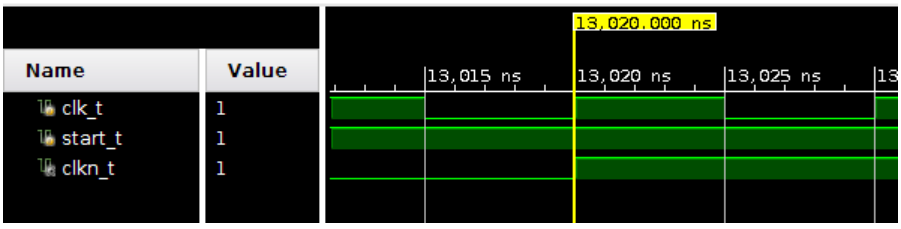
initial
begin
start_t=1'b0;
clk_t=1'b0;
#5;
start_t=1'b1;
```

forever
#5 clk_t=~clk_t;
end
endmodule

The time between two consecutive rising edges of clkn_t is 8680ns. Its frequency is therefore ~115200 Hz.







Part 3

module receive(start, clkn, rx, bits, done);
input clkn;
input start;
input rx;
output reg [8:0] bits;
output reg done;
reg [3:0] counter;

initial
begin
bits=9'd0;
done=1'b0;
counter=4'd0;
end

always @(posedge clkn)

```
begin
   if (start == 1'b1 && done==1'b0)
   begin
      if(counter==4'd9)
      begin
         done=1'b1;
         counter=4'd0;
      end
      else
      begin
         counter=counter+1'b1;
         bits[9-counter]=rx;
      end
   end
end
endmodule

`timescale 1ns / 1ps

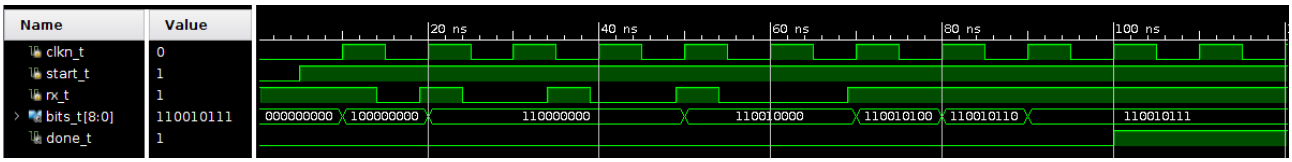
module receive_tb;
reg clkn_t;
reg start_t;
reg rx_t;
wire [8:0] bits_t;
wire done_t;

receive u1(start_t, clkn_t, rx_t, bits_t, done_t);

initial
begin
start_t=1'b0;
clkn_t=1'b0;
#5;
start_t=1'b1;
repeat (32)
#5 clkn_t=~clkn_t;
$stop;
end

initial
begin
//sending inputs in a random sequence..
rx_t=1'b1; #4;
rx_t=1'b1; #5;
rx_t=1'b1; #5;
rx_t=1'b0; #5;
rx_t=1'b1; #5;
rx_t=1'b0; #5;
```

```
rx_t=1'b0; #5;
rx_t=1'b1; #5;
rx_t=1'b0; #5;
rx_t=1'b0; #5;
rx_t=1'b1; #5;
rx_t=1'b0; #5;
rx_t=1'b0; #5;
rx_t=1'b0; #5;
rx_t=1'b1; #5;
rx_t=1'b1; #5;
end
endmodule
```



Part 4

```
module LED_turn_on_off(clk, bits, done, led);
input clk;
input [8:0] bits;
input done;
output reg [0:0] led;

wire parity;
assign parity=bits[8]^bits[7]^bits[6]^bits[5]^bits[4]^bits[3]^bits[2]^bits[1];

initial
led[0]=1'b0;

always @(posedge clk)
begin
   if(done==1'b1)
   begin
      if(parity==bits[0])
      begin
         if(bits[8:1]==8'h4F)
            led[0]=1'b1;
         else if(bits[8:1]==8'h43)
            led[0]=1'b0;
      end
      else
         led[0]=1'b0;
   end
end
endmodule
```

```
`timescale 1ns / 1ps

module LED_turn_on_off_tb;
reg clk_t;
reg [8:0] bits_t;
reg done_t;
wire led_t;

LED_turn_on_off u1(clk_t, bits_t, done_t, led_t);

initial
begin
clk_t=1'b0;
done_t=1'b0;
#5;
done_t=1'b1;
repeat(10)
#5 clk_t=~clk_t;
$stop;
end

initial
begin
bits_t={8'h55,1'b0};#13; // wrong ascii
bits_t={8'h4f,1'b0};#10; // ascii "O" but wrong parity
bits_t={8'h4f,1'b1};#10; // ascii "O" true parity
bits_t={8'h43,1'b0};#10; // ascii "C" but wrong parity
bits_t={8'h43,1'b1};#10; // ascii "C" true parity
end
endmodule
```

